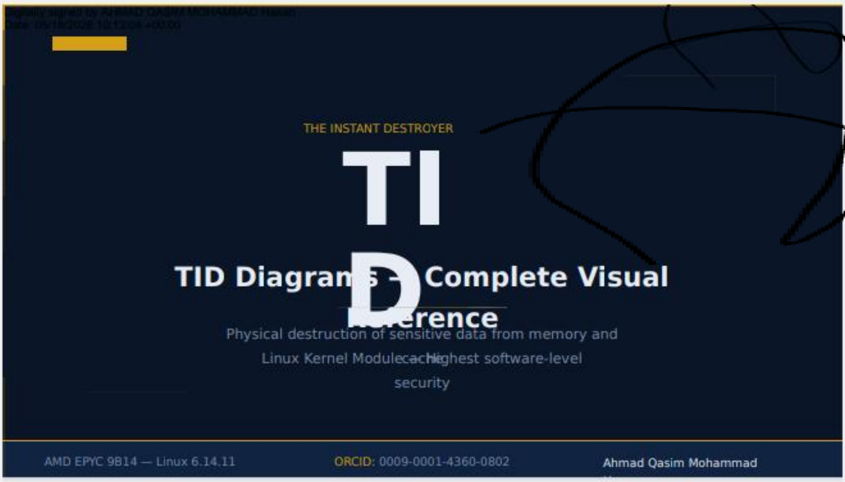
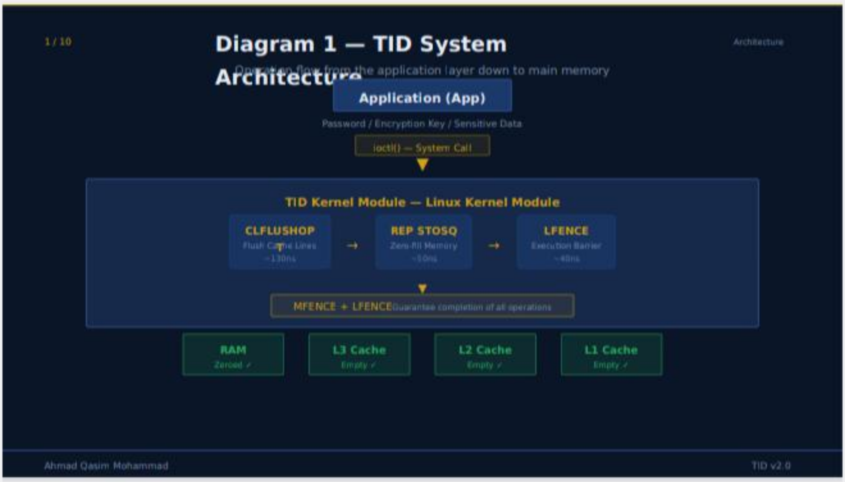

THE INSTANT DESTROYER
TID
TID Diagrams — Complete Visual Reference
Physical destruction of sensitive data from memory and cache
Linux Kernel Module — Highest software-level security
AMD EPYC 9B14 — Linux 6.14.11
ORCID: 0009-0001-4360-0802
Ahmad Qasim Mohammad


1 / 10
Diagram 1 — TID System Architecture
Architecture
Operation flow from the application layer down to main memory
Application (App)
Password / Encryption Key / Sensitive Data
ioctl() — System Call
TID Kernel Module — Linux Kernel Module
CLFLUSHOP
REP STOSQ
LFENCE
MFENCE + LFENCE
Guarantee completion of all operations
RAM
L3 Cache
L2 Cache
L1 Cache
Ahmad Qasim Mohammad
TID v2.0


2 / 10
Diagram 2 — PROTECT+ZERO Protocol
Sensitive data lifecycle — three phases
ZERO Phase
After data use
Block speculation again LFENCE
Write zeros to memory REP STOSQ
Evict cache L1+L2+L3 CLFLUSHOPT
Ensure flush completion MFENCE
Close the second side LFENCE
Data in Use
Key / Password / Data
Attack Window
372 ns
Physical hardware minimum
PROTECT Phase
Before data use
LFENCE Block speculative execution
MFENCE Ensure prior writes complete
✓ Attacker finds nothing
✓ Cache fully empty
✓ Memory is clean
Ahmad Qasim Mohammad
TID v2.0

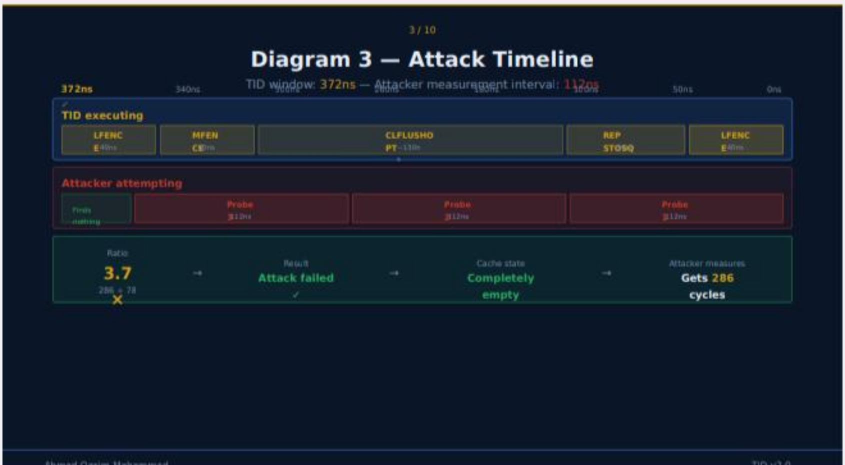

3 / 10
Diagram 3 — Attack Timeline
TID window: 372ns — Attacker measurement interval: 112ns
372ns
340ns
50ns
0ns
TID executing
LFENCE
MFENCE
CLFLUSHOPT
REP STOSQ
LFENCE
Attacker attempting
Probe
Probe
Probe
Ratio
3.7
Result
Attack failed
Cache state
Completely empty
Attacker measures
Gets 286 cycles

Diagram 4 — Cache Comparison: Before and After TID

Three scenarios — No protection / TID v1 / TID v2

✓ TID v2.0	⚠ TID v1.0	✗ No TID	Cache Layer
░░░░ (fully empty)	0xAB... (key present)	0xAB... (key present)	L1 Cache
░░░░ (fully empty)	0xAB... (key present)	0xAB... (key present)	L2 Cache
░░░░ (fully empty)	Zeroed (partial wipe)	0xAB... (key present)	L3 Cache
Nothing ✓ Secure	Partial data ✗	The data ✗ Danger	Attacker finds

* v1.0 wipes main memory but leaves cache populated — the vulnerability remains exploitable via Flush+Reload

Diagram 5 — Performance Comparison

CPU cycles to access data — More cycles = More secure

Cache HIT
78 cycles
Direct leak ✗

Improvement ratio
3.7×
286 ÷ 78

Cache MISS
286 cycles
Attack blind ✓

Attacker hits **RAM** not cache — attack fails

Diagram 6 — TID vs Existing Libraries

Security capabilities across industry solutions

Verdict	LFENCE Barrier	[illegible]	[illegible]	Library / Solution
Unreliable	✗	✗	✗ *compiler may remove	memset()
Partial	✗	✗	✓	explicit_bzero()
Partial	✗	✗	✓	memzero_explicit()
Partial	✗	✗	✓	OPENSSL_cleanse()
Partial	✗	✗	✓	sodium_memzero()
Best software ✓	✓	✓	✓	► TID v2.0 ◄
Requires dedicated hardware	Automatic ✓	Automatic ✓	✓	Intel SGX (hardware)

* memset() is optimized away by the compiler in many cases — a documented C/C++ vulnerability

Diagram 7 — Statistical Key Reconstruction Attack

Flush+Reload attack — TID v1 vs TID v2

Phase 2 — TID v2

Cache always empty — attack fails

Slot 13	Slot 133	Slot 87	Slot 42	Probe
✗ empty	✗ empty	✗ empty	✗ empty	Probe 1
✗ empty	✗ empty	✗ empty	✗ empty	Probe 500
✗ empty	✗ empty	✗ empty	✗ empty	Probe 1000

Statistical accumulation

0/1000	0/1000	0/1000	0/1000
no pattern	no pattern	no pattern	no pattern

No pattern — No key — Attack failed ✓
Cache empty — nothing to measure

Phase 1 — TID v1

Cache not flushed — attack succeeds

Slot 13	Slot 133	Slot 87	Slot 42	Probe
✗ absent	✓ present	✓ present	✓ present	Probe 1
✗ absent	✓ present	✓ present	✓ present	Probe 500
✗ absent	✓ present	✓ present	✓ present	Probe 1000

Statistical accumulation

3/1000	978/1000	991/1000	987/1000
not in key	key fragment	key fragment	key fragment

Key reconstructed byte by byte ✗
Successful attack on TID v1

Diagram 8 — Security Solution Ladder

Weakest to strongest — solution ranking

Automatic ✓ Cache flush	**Intel SGX / AMD SEV** Dedicated hardware — Encrypted cache — Zero attack window	① Dedicated Hardware
Explicit ✓ Cache flush	**► TID v2.0 ◄** ← You are here Wipe + CLFLUSHOPT + LFENCE/MFENCE — highest software level without dedicated hardware	② Best Software
✗ Missing Cache flush	**libsodium / OpenSSL / glibc** Wipe of main memory only	③ Partial Software
✗ Missing Cache flush	**memset() standard** Compiler may remove — no actual guarantee	④ Unreliable

TID v2.0 is **the maximum achievable in software** without dedicated hardware — proven experimentally

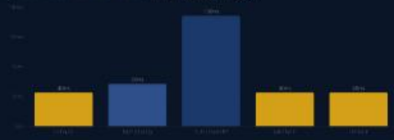
Diagram 9 — Physical Minimum Latency of TID

Per-instruction timing — a physical constraint of the CPU itself

~40ns	**LFENCE** Execution barrier — blocks speculative execution Position: Start
~50ns	**REP STOSQ** Write zeros — wipe main memory Position: Wipe
~130ns	**CLFLUSHOPT** Evict cache lines from L1/L2/L3 Position: Cache flush
~40ns	**MFENCE** Guarantee all operations complete Position: Completion
~40ns	**LFENCE** Close the second side Position: End

The constraint is **CPU physics** itself, not code efficiency — measured value: 290–372ns

Time in nanoseconds (ns)



Total **300-372** ns

Physical hardware minimum — proven experimentally

Diagram 10 — Experimental Proof Summary

AMD EPYC 9B14 | Linux 6.14.11 | GCC 14.3.0 | 2599 MHz | 62.8 GiB RAM | -O2 -mclflushopt

Test 3 — CPU Shield (5,000 samples)

- CPU Affinity ✓
- mlock ✓
- SCHED_FIFO requires root

Test 2 — Timing Window (10,000 samples)

- TID window **372ns**
- Attacker probe **112ns**
- Possible probes **3.3 only**

Test 1 — Basic Attack (1,000 samples)

Ratio	v2	v1
3.7×	286 cycles	78 cycles

Experimentally Verified Final Result

Maximum achievable in software without SGX/SEV	Attacker measures and finds zero — no pattern — no key	TID v2.0 defeats Flush+Reload after use

TID INSTANT DESTROYER

Security Level	Protection Ratio	Destruction Window
② Maximum software level	3.7× Attacker access overhead	372ns Physical hardware minimum

Ahmad Qasim Mohammad Hassan

TID Project — 2026 ORCID: 0009-0001-4360-0802

Cache empty — Memory clean — Attacker finds nothing